EXTENSION TO THE PIPELINE BUILD SPEC · PLANE I

First-Party Data Plane

Three assets already exist. None of them is what its name suggests — the job data is not a signal, the firm data is not a universe, and the portal is not just a product. Placed correctly, one of them closes a gap the build spec had left open.

Adds plane I · 4 components **Changes** arming gate, fit score **Closes** the supply gate

§1 What we actually have

Each asset has an obvious use and a correct use, and for two of the three they are not the same. Getting this classification wrong is what turns useful internal data into contamination.

ASSET → WHAT IT LOOKS LIKE → WHAT IT IS

ASSET	LOOKS LIKE	ACTUALLY IS	ENTERS AS
Job-posting data companies that posted roles — no post link, no company URL	An intent signal. "These firms were hiring."	**A seed list plus a fit attribute.** Without a link there is no way to know whether the role is still open, and an unverifiable posting is not a signal — it is a lead on where to look. Its durable value is different and better: it shows which firms *structurally* hire support staff.	crawl seed + fit attribute
Firm dataset firm records, size, attributes	A ready-made universe. "We already have the list."	**A universe accelerator and an attribute source — not a universe.** It cannot contain the field the whole ICP turns on, because no commercial dataset carries fee model. It shortens discovery; it does not replace qualification.	observation + firm_attribute
Portal talent pool + daily job postings	The product.	**Two feeds that belong at opposite ends of the funnel.** The talent pool is the supply side of the arming gate — the table the build spec specified but had no source for. The role requests are first-party demand, and they must reach suppression before they reach outreach.	bench_inventory + firm_track

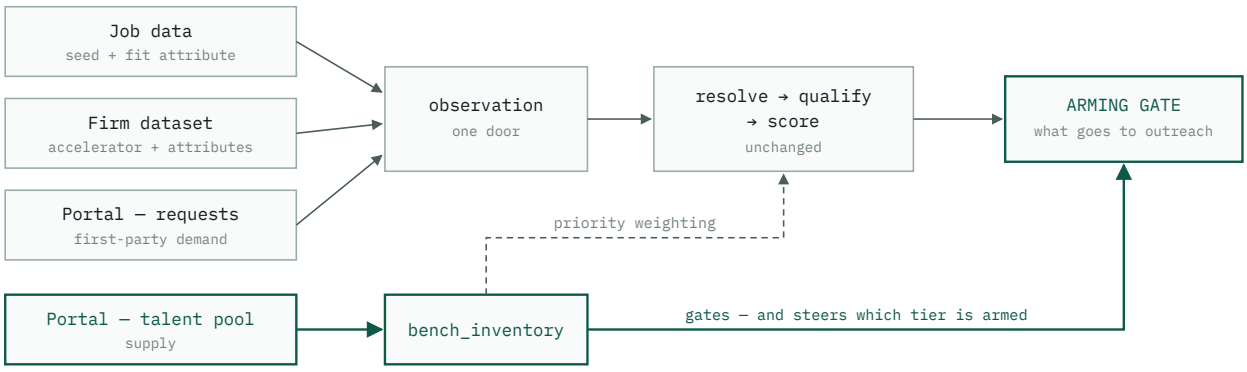


Fig 1 — Three of the four feeds enter through the same door as every external source, so nothing bypasses resolution or reconciliation. The fourth runs down a separate path to `bench_inventory` and rejoins at the arming gate — the only place in the system where supply and demand meet.

§2 One door

Internal data feels trustworthy, which makes it tempting to load it straight into `firm` and skip the pipeline. Do not. Every record enters through `observation` with a `source` tag, exactly like a scraped one.

REASON	WHAT BREAKS WITHOUT IT
Provenance	When a firm is scored into the wrong tier six weeks from now, there is no way to tell whether the cause was bad internal data, a bad match, or a bad extraction. Every debugging session becomes archaeology.
Reconciliation	Internal data will disagree with the firm's own website about size, name and phone. Loaded directly, it silently wins or silently loses. Through `firm_attribute`, the precedence rules in the build spec decide, and the decision is inspectable.
Reversibility	If a bulk load turns out to be mismatched, an `observation`-sourced load can be withdrawn by source tag. A direct write cannot be separated from the rest.
Freshness	Internal data has an age, often an unknown one. Through the normal path it inherits the 90-day rules and gets re-verified. Loaded directly it looks permanently current.

> THE ONE EXCEPTION
>
> `bench_inventory` is not firm data and does not go through `observation`. It is a live mirror of internal state with its own sync, its own freshness requirement, and no identity resolution to do. It is the only feed in the system that writes to a table directly.

§3 Job data — a seed, not a signal

A job posting with no link cannot be verified. We cannot tell whether the role is still open, and "still open after three weeks" is the entire reason that signal is the strongest one on the board. Treating unverifiable history as live intent would send outbound at firms that filled the role months ago.

So it does not enter as a signal. It does two other things, and the second is more valuable than the first.

Use one — seed the crawl queue

Every company name in the file is a candidate that may or may not already be in the universe. Resolve it, and if it is new, it enters discovery. If it resolves to a firm we already have, the record adds nothing to the universe but everything to use two.

Use two — structural hiring behaviour

This is the durable value. A firm that has posted support roles in the past has **proved it has that work and has had budget for it.** That is a fit attribute, not an intent one, and it survives indefinitely.

```
-- new fit attributes derived from historical job data

  has_hired_support_roles     bool     -- ever posted a support role
  support_roles_posted        text[]   -- which ones: paralegal, intake…
  last_known_hiring_attempt   date
  repeat_role_count           int      -- same role posted N times

-- FIT contribution
  has_hired_support_roles                              +5
  repeat_role_count >= 2   strongest of the four       +8

-- a firm that has posted "paralegal" three times in two years has
-- churn in that seat. the need is not occasional, it is chronic —
-- and chronic need in a seat we can fill permanently is a better
-- prospect than a firm hiring for the first time. this pattern is
-- only visible with historical data. it is the single best reason
-- to keep this file.

-- what it must NOT do
  never sets intent_score          -- unverifiable
  never arms a lead                -- §10 arming needs a live signal
  never appears in outreach copy   -- "you posted a role in March"
                                   -- is a liability, not a first line
```

Whether it can be matched at all

Everything above depends on resolving a company name to a firm entity. Without a URL that is the weakest match rule in the system. Run this triage on the file before planning around it — the answer determines whether the asset is usable, partly usable, or a trap.

MATCH LADDER · CHECK WHICH FIELDS THE FILE ACTUALLY CARRIES

FIELDS PRESENT	MATCH CONFIDENCE	HANDLING
Name + any URL or email domain	DECISIVE	Auto-merge on rule R1. Fully usable.
Name + phone	DECISIVE	Auto-merge on rule R2.
Name + full street address	STRONG	Rule R3 — address hash plus name similarity. Usable.
Name + city/state only	PROBABILISTIC	Rule R4 territory. **Never auto-merge.** Route to the review queue, or accept it only where exactly one candidate exists in that city above the similarity threshold.
Name only	UNUSABLE FOR MATCHING	Do not attempt. "Smith Law" resolves to dozens of firms nationally, and a wrong merge writes a false hiring history onto a real firm permanently.

> **BAD INTERNAL DATA IS WORSE THAN NONE**
> A mismatched merge does not fail loudly. It writes a plausible attribute onto the wrong firm, which then scores wrongly, routes wrongly, and gets contacted with a premise that is not true — forever, because nothing downstream will ever question it. Tag every attribute from this source and keep it withdrawable.

§4 Firm dataset — accelerator, not universe

This is the strongest of the three assets and the one most likely to be over-trusted. It can carry a firm most of the way through discovery, but it cannot carry it through qualification, because **the field the entire ICP turns on is not in any commercial dataset.**

WHAT IT CAN AND CANNOT REPLACE

ATTRIBUTE	DATASET	STILL REQUIRED
Firm name, address, phone	USABLE	Reconciled against site and Places by precedence
Domain	USABLE — THE KEY FIELD It is the strongest match key and the entry point to the crawl	—
Firm size	USABLE	Cross-check against team page. Vendor size fields are often headcount, not attorney count — a material difference here.
Practice area	COARSE	Site classification still runs. Vendor taxonomies do not distinguish PI from general litigation.
Fee model	ABSENT	**Site crawl, always.** No shortcut exists.
Staff ratio, languages, case system	ABSENT	Site crawl
Named decision-maker	SOMETIMES	Verify against bar records and the team page. Commercial data on small firms goes stale fast.

First task: coverage analysis

Before deciding how much discovery work this saves, measure the overlap. This is one query and it changes the plan.

```
-- run after loading the dataset as observations and resolving

  in dataset only        →  firms the external harvest would miss
  in Places only         →  the dataset is not complete
  in both                →  the overlap
  dataset ∩ ICP tiers    →  how much of it is even relevant

-- what each outcome means
  high overlap, low ICP share   → dataset is broad but off-target.
                                  use it for attributes, keep full harvest.
  low overlap, high ICP share   → genuinely additive. harvest still needed
                                  for coverage, but this is a strong seed.
  high overlap, high ICP share  → discovery can be narrowed to gap-fill.
                                  the biggest possible saving.
```

Precedence

Where this data sits in the reconciliation order depends on where it came from, and that determination has to be made before the first load rather than after the first conflict.

ORIGIN	RANK IT	BECAUSE
Built by the team from real interactions	Above Places, below the firm's own site	Human-verified, but ages without anyone noticing
Purchased from a data vendor	Directory tier — below Places	Same staleness profile as Justia or Avvo
Unknown provenance	Lowest, and flagged	Unknown age plus unknown method is not a source, it is a hint

§5 The portal — two feeds, opposite ends

The portal is the only genuinely proprietary asset here. Everything else could in principle be reconstructed from public sources; this could not. It also carries the highest risk of an embarrassing failure if it is wired in late.

Feed A — the talent pool

This is `bench_inventory`. The build spec specified that table and the gate that reads it, but had no source for it. The portal is the source, which turns a hypothetical constraint into a live one.

```
-- bench_inventory, sourced from the portal talent pool
  sku            demand_writer | paralegal | intake | case_manager | …
  language       en | es | pt | …
  practice_exp   personal_injury | immigration | family | …   ← matters
  available      int
  replenish_30d  int
  as_of          timestamptz

-- the practice_exp dimension is what makes this useful rather than
-- decorative. "12 case managers available" tells the arming gate
-- almost nothing. "12 case managers, 4 of them Spanish-speaking with
-- immigration experience" tells it exactly which tier to arm.
```

Feed B — the daily role requests

Whatever these turn out to be, they resolve to firms, and firms that appear here must not be treated as cold. The handling differs by which of two things the feed actually is.

IF THE FEED IS…	THEN THOSE FIRMS ARE	ROUTE TO
Law firms posting roles they want filled the likely reading	Either existing customers or self-identified prospects. Either way they have raised their hand. **The hottest population in the system** — and the one it would be most damaging to cold-email.	firm_track = 'portal' suppress from cold sequences route to the human who owns the account
Externally aggregated postings collected for internal use	The same class of data as §3 — and if these carry links and dates, they are a genuine live signal rather than a seed.	signal kind='job_posting_open' only where a URL and date exist

> **WIRE SUPPRESSION BEFORE ANYTHING ELSE**
> If the first reading is right, a cold sequence reaching a firm that already posted a role on our own portal is the worst single failure this system can produce. It is visible to the customer, it contradicts the product, and it costs an account. The portal-to-suppression path should be connected before the first outbound email is ever sent — ahead of scoring, ahead of signals, ahead of everything.

Feed B has a third use

The role requests are also **ground truth for the tier → wedge-SKU table.** That mapping in the build spec is a set of reasoned guesses. Portal data replaces it with fact: what kind of firm actually asks for what role, first, in their own words. No amount of outbound testing produces that as cleanly.

§6 Supply steers demand

The build spec used the bench as a brake — do not arm a lead for a role we cannot fill. With a live feed it can do something more useful: decide *what to go looking for this week.*

The standard model generates demand and then hopes delivery can match it. Inverted, the bench becomes an input to targeting rather than a constraint discovered at the end:

- Fourteen Spanish-speaking legal assistants with immigration experience sitting available → **this week's outbound weights Tier 1B up.**
- Six demand writers with PI experience → Tier 1A on the demand-writer wedge gets priority.
- Zero lien negotiators, none replenishing → that wedge is paused in messaging, not merely gated at the end.

```
-- arming threshold becomes a function of supply

  threshold(tier, sku) =
      BASE                                  -- 60
    - bench_bonus(available, replenish_30d) -- deep bench: arm earlier
    + starvation_block(available)           -- empty: do not arm at all

  bench_bonus     = LEAST(15, available × 1.5)
  starvation_block= CASE WHEN available = 0
                         AND replenish_30d = 0 THEN 999 ELSE 0 END

-- a deep bench lowers the bar, so borderline leads for a role we can
-- comfortably fill still get worked. an empty bench with no pipeline
-- raises it out of reach. the gate stops being binary and starts
-- behaving like inventory pressure.
```

THE BENCH IS ALSO A MESSAGE

Because the talent pool is first-party and current, outreach can name real inventory. *"We have a paralegal with three years of personal injury experience available this week"* is a materially different opening from *"we place paralegals"* — it is specific, it is checkable, and it implies immediacy that a generic pitch cannot.

No competitor working from public job-posting data can write that line, because none of them knows what is on their bench at the moment they send. This is the one place where an internal asset produces an advantage that is not merely operational.

§7 Plane I components

Four new components, added alongside D, P, W and R in the build spec's register.

PLANE I — FIRST-PARTY SOURCES				
ID	COMPONENT	INPUT → OUTPUT	CADENCE	NOTES
I1	**Firm dataset loader**	`export` → `observation`	One-shot, re-runnable	Source tag `internal_firms`. Runs the coverage analysis as part of the load and reports it.
I2	**Job history loader**	`export` → `observation` + `firm_attribute`	One-shot, re-runnable	Applies the match ladder. Anything at name-only confidence is rejected, not merged. Writes fit attributes only — never signals.
I3	**Portal demand feed**	`portal` → `firm_track` + `suppression`	Hourly	**Build first.** Suppression correctness matters more than latency anywhere else in the system.
I4	**Talent pool sync**	`portal` → `bench_inventory`	Hourly	Writes directly — the one feed that does not pass through `observation`. Stale bench data is worse than none: it re-opens the exact failure the gate exists to prevent.

CHANGES TO EXISTING COMPONENTS

- **Fit score** gains `has_hired_support_roles` (+5) and `repeat_role_count ≥ 2` (+8).
- **Arming gate** takes a supply-derived threshold instead of a fixed 60.
- **Suppression check** gains the portal track — and is checked at send time, not at arm time.
- **Tier → wedge SKU** becomes a table fed by portal request data rather than by seeded guesses.

§8 Risks

RISK	HOW IT SHOWS UP	GUARD
Cold-emailing a portal firm SEVERE	A customer receives a prospecting email from the company they already buy from. Contradicts the product, in writing, to the person paying.	I3 before any outbound. Suppression at domain scope, checked at send.
Name-only merges from job data	Silent. A false hiring history lands on a real firm and is never questioned again.	Match ladder. Reject at name-only confidence rather than guessing.
Historical postings treated as live	Outbound opens with a role the firm filled eight months ago. Reads as careless and kills the reply.	Job history writes fit attributes only. Intent requires a verifiable live source.
Over-trusting the firm dataset	Discovery is narrowed on the assumption the dataset is complete, and a segment quietly never gets collected.	Coverage analysis before any scope reduction. Numbers, not assumption.
Stale bench data	Leads arm against inventory that is already placed. The three-day promise breaks on exactly the leads the system worked hardest to find.	Hourly sync, and an alert when `as_of` falls behind. Treat a stale bench as empty.
Vendor size fields misread	Total headcount taken as attorney count. Firms land in the wrong band and the 26–50 penalty misfires.	Cross-check against team page on a sample before trusting the field.

§9 Order of work

Ordered by risk retired per unit of effort, not by data volume.

#	DO	WHY FIRST
1	**I3 — portal demand → suppression**	Retires the only failure here that damages an existing customer. Small job, and nothing outbound should run before it exists.
2	**I4 — talent pool → bench_inventory**	Closes the supply gate the build spec left open, and unlocks §6. Also small.
3	**Triage the job data against the match ladder**	One inspection that determines whether the asset is usable at all. Cheap, and it prevents planning around data that cannot be matched.
4	**I1 — firm dataset load + coverage analysis**	Tells us how much discovery work is actually needed. Changes the scope of the largest stage in the build.
5	**I2 — job history load**	Last, because its value is a fit-score refinement rather than a gate, and it depends on step 3's verdict.

> WHAT THIS CHANGES ABOUT THE BUILD
>
> Steps 1 and 2 are both small and both should land before the first outbound send. Step 4 may materially shrink the discovery plane — which is the largest and most expensive stage in the whole system — so it is worth doing early enough to influence that scope rather than after it has been built to full size.

Extension to the Pipeline Build Spec. Adds plane I (four components), changes the arming gate from a fixed threshold to a supply-derived one, and adds two fit attributes.

Handling of the portal role-request feed depends on which of the two readings in §5 is correct; the suppression path is required under either. Weights and thresholds are seeded values to be replaced by measurement.